# Reflection and Traceability Report on Audio360

Team #6, Six Sense
Omar Alam
Sathurshan Arulmohan
Nirmal Chaudhari
Kalp Shah
Jay Sharma

# 1 Symbols, Abbreviations, and Acronyms

| symbol | description |
|---|---|
| DoA | Direction of Arrival |
| Doc | Document |
| FR | Functional Requirement |
| HA | Hazard Analysis |
| MG | Module Guide |
| MIS | Module Interface specification |
| NFR | Non-Functional Requirement |
| SRS | software Requirement specification |
| TA | Teaching Assistant |
| VnV | Verification and Validation |

Table 1: Symbols, abbreviations and acronyms used in the VnV plan document.

See SRS Documentation at Symbols, Abbreviations, and Acronyms for a complete table used in Audio360.

# Contents


**1 Symbols, Abbreviations, and Acronyms** **2**

**2 Changes in Response to Feedback** **4**
2.1 SRS and Hazard Analysis . . . 4
2.2 Design and Design Documentation . . . 5
2.3 VnV Plan and Report . . . 7

**3 Extras** **9**

**4 Design Iteration (LO11 (PrototypeIterate))** **9**

**5 Design Decisions (LO12)** **13**
5.1 Limitations . . . 13
5.2 Constriants . . . 14
5.3 Assumptions . . . 14

**6 Economic Considerations (LO23)** **14**

**7 Reflection on Project Management (LO24)** **14**
7.1 How Does Your Project Management Compare to Your Development Plan . . . 14
7.2 What Went Well? . . . 15
7.3 What Went Wrong? . . . 15
7.4 What Would you Do Differently Next Time? . . . 15

**8 Ethics and Equity (LO18)** **16**

**9 Reflection on Capstone** **16**
9.1 Which Courses Were Relevant . . . 17
9.2 Knowledge/Skills Outside of Courses . . . 18


# List of Tables


1 Symbols, abbreviations and acronyms used in the VnV plan document. . . . 2
2 SRS Feedback Tracking and Implementation . . . 5
3 Hazard Analysis Feedback Tracking and Implementation . . . 5
4 MG Documentation Feedback Tracking and Implementation . . . 6
5 MIS Documentation Feedback Tracking and Implementation . . . 7
6 VnV Plan Feedback Tracking and Implementation . . . 8
7 VnV Report Feedback Tracking and Implementation . . . 9


# List of Figures


1 Hardware set-up for Proof of Concept. . . . 10
2 Final hardware. . . . 11
3 Initial UI before user testing. . . . 12
4 UI after user testing. . . . 13

# 2 Changes in Response to Feedback

Feedback source that lists "team" indicates that the change request was made from one of team members of this project through discussions during team meetings or during development.

Any feedback with "WONT FIX" label for Changes Made column means that the team will not be addressing the feedback. The rationale for not addressing will be in the GitHub issue that is associated to the feedback.

No code changes resulted directly from feedback provided by peers, the TA, or the supervisor. Peers and the TA were not responsible for reviewing code, and feedback on documentation did not influence the code or development process. Additionally, the supervisor did not have further input related to code or development, as the team had already demonstrated a well-functioning product early in the capstone. It is important to note, however, that the team made several internal modifications to the code throughout the project. These changes arose from experimentation and testing rather than from external feedback or documentation revisions.

## 2.1 SRS and Hazard Analysis

Table 2.1 outlines all the changes for the SRS document. Table 2.1 outlines all the changes for the HA document.

| **Feedback Source** | **Feedback Summary** | **Changes Made** | **Related Commits/Issues** |
|---|---|---|---|
| Peer | Add technical terms to glossary. | SRS Doc version 2.0 | Commit 6ba4c40<br>Issue #224 |
| Peer | Missing Context diagram. | SRS Doc version 2.1 | Commit f31adb6<br>Issue #223 |
| Supervisor | Decrease sampling rate for feasibility. | SRS Doc version 2.2 | Commit 56ba2bc<br>Issue #243 |
| TA | Define symbols for constants + rationale. | SRS Doc version 3.1 | Commit 8ea30f2<br>Issue #284 |
| TA | Cite 1 in 10 Canadians with hearing loss. | SRS Doc version 3.1 | Commit 8ea30f2<br>Issue #284 |
| TA | Cite 1 in 10 Canadians with hearing loss. | SRS Doc version 3.1 | Commit 8ea30f2<br>Issue #284 |
| TA | Removed discomfort invariant from User Safety. | SRS Doc version 3.1 | Commit 8ea30f2<br>Issue #284 |
| TA | Component Diagram formatting. | SRS Doc version 3.1 | Commit 8ea30f2<br>Issue #284 |
| TA | Functional and Non-Functional requirements lack rationale. | SRS Doc version 3.1 | Commit 8ea30f2<br>Issue #284 |
| TA | Component Diagram formatting. | SRS Doc version 3.2 | Commit 8ea30f2<br>Issue #284 |
| TA | VnV information is too detailed for SRS. | SRS Doc version 3.3 | Commit 8ea30f2<br>Issue #284 |
| Peer | Redundant documentation of team roles | SRS Doc updated | Commit 7300b3b<br>Issue #222 |

Continued on next page

**Table 2 continued from previous page**

| Feedback Source | Feedback Summary | Changes Made | Related Commits/Issues |
|---|---|---|---|
| Peer | Missing Traceability between FR and NFR | WONT FIX | Issue #220 |
| Peer | Abstract sampling rate | WONT FIX | Issue #219 |
| Team | Software layer abstraction requirement | SRS Doc version 4.0 | Commit 4319be3 |
| Team | Missing symbols, abbreviations, and accronym table | SRS Doc version 4.1 | Commit 286836e |
| TA | Missing symbols, abbreviations, and accronym table | SRS Doc version 4.1 | Commit 286836e<br>Issue #337 |
| Team | Add source for angle accuracy for DoA | SRS Doc version 4.2 | Commit 4c69951 |
| Peer | Convert all angle representation to radians. | SRS Doc version 4.3 | Commit 9ae79c4<br>Issue #383 |
| Team | Specify classification classes | SRS Doc version 5.1 | Commit 6edf8cc<br>Issue #624 |
| Team | Minimal user input requirement needed | SRS Doc version 5.2 | Commit f139591<br>Issue #650 |

Table 2: SRS Feedback Tracking and Implementation

| Feedback Source | Feedback Summary | Changes Made | Related Commits/Issues |
|---|---|---|---|
| Supervisor | Decrease sampling rate for feasibility. | HA Doc version 2.0 | Commit 56ba2bc<br>Issue #243 |
| TA | Missing rationale for roadmap. | HA Doc version 2.1 | Commit e47be90<br>Issue #285 |
| TA | Ambiguous lighting condition definition. | HA Doc version 2.2 | Commit e47be90 |
| Peer | Safety requirements to SRS requirement traceability. | HA Doc version 2.3 | Commit e47be90<br>Issue #221 |
| TA | Missing analysis on comfort/harm. | HA Doc version 2.4 | Commit 94d7bc1<br>Issue #286 |
| TA | Missing link between FMEA table and safety requirements. | HA Doc version 2.4 | Commit 94d7bc1<br>Issue #286 |

Table 3: Hazard Analysis Feedback Tracking and Implementation

## 2.2 Design and Design Documentation

Due to strong planning ahead of time, there was no feedback received towards the design of the project. The team also did not update the design after the initial write up of the design document. Thus, table 2.2 outlines the feedback in regards to the MG design document, and table 2.2 outlines the feedback in regards to the MIS design document.

| Feedback Source | Feedback Summary | Changes Made | Related Commits/Issues |
|---|---|---|---|
| Peer | Clarification on closed system | MG Doc updated | Commit 457d8ed<br>Issue #381 |
| Peer | Convert all angle representation to radians. | MG Doc updated | Commit 9ae79c4<br>Issue #383 |
| Peer | Exported constants traceability. | MG Doc updated | Commit 1643151<br>Issue #384 |
| Peer | AC missing from traceability table. | WONT FIX | Issue #386 |
| Peer | Module type clarity. | MG Doc version 2.0 | Commit a67ab08<br>Issue #387 |
| Peer | Document anticipated changes. | WONT FIX | Issue #388 |
| Peer | MG split SPI and file system. | WONT FIX | Issue #390 |
| Peer | Define "Output" in Use Hierarchy. | MG Doc updated | Commit 28bb900<br>Issue #399 |
| TA | Detailed timelines required. | MG Doc version 2.1 | Commit 766437e<br>Issue #569 |
| TA | Rationale need for Anticipated Change 6. | MG Doc updated | Commit 9b8a467<br>Issue #570 |
| TA | Timeline for individual contributions. | MG Doc version 3.0 | Commit 9b8a467<br>Issue #570 |
| TA | Uses hierarchy diagram simplification. | MG Doc version 3.1 | Commit 9b8a467<br>Issue #570 |
| TA | Remove safety critical information logic from Visualization module. | MG Doc version 3.2 | Commit 9b8a467<br>Issue #570 |
| TA | Spelling and grammar fixes. | MG Doc version 3.3 | Commit 9b8a467<br>Issue #570 |
| TA | Add module names to module hierarchy table. | MG Doc version 3.4 | Commit 9b8a467<br>Issue #570 |

Table 4: MG Documentation Feedback Tracking and Implementation

| Feedback Source | Feedback Summary | Changes Made | Related Commits/Issues |
|---|---|---|---|
| Peer | Clearer assumptions for many modules. | MIS Doc version 2.0 | Commit bceb3ca<br>Issue #382 |
| Peer | Convert all angle representation to radians. | MIS Doc updated | Commit 9ae79c4<br>Issue #383 |
| Peer | Exported constants traceability. | MIS Doc updated | Commit 1643151<br>Issue #384 |

Continued on next page

**Table 5 continued from previous page**

| Feedback Source | Feedback Summary | Changes Made | Related Commits/Issues |
|---|---|---|---|
| Peer | Duplicated Audio360State definition. | MIS Doc updated | Commit d10e851<br>Issue #385 |
| Peer | Modules are too low-level. | WONT FIX | Issue #389 |
| TA | Fault Manager module should be more detailed. | MIS Doc version 3.0 | Commit 34101a8<br>Issue #568 |
| Team | Visualization module should be more detailed. | MIS Doc version 3.1 | Commit 34101a8 |
| TA | DoA formalation. | MIS Doc version 3.2 | Commit f290eb6<br>Issue #567 |

Table 5: MIS Documentation Feedback Tracking and Implementation

## 2.3 VnV Plan and Report

Table 2.3 outlines all the changes for the VnV Plan document. Table 2.1 outlines all the changes for the VnV Report document.

| Feedback Source | Feedback Summary | Changes Made | Related Commits/Issues |
|---|---|---|---|
| Peer | Ambiguious latency definition. | VnVPlan version 2.0 | Commit 3fc34af<br>Issue #261 |
| Peer | Missing pass fail criteria for some tests. | VnVPlan version 3.2 | Commit 4c69951<br>Issue #262 |
| Peer | Add testing and validation frameworks. | VnVPlan version 2.1 | Commit 09aa9f3<br>Issue #263 |
| Peer | Add measureable validation thresholds for user testing. | VnVPlan version 2.4 | Commit 446ca14<br>Issue #264 |
| Peer | Split SRS verification and validation. | VnVPlan updated | Commit e6b0fa8<br>Issue #266 |
| Peer | Mismatch of NFR in VnV Plan tests. | VnVPlan updated | Commit 5ee1b30<br>Issue #265 |
| Peer & TA | Clearer design verification checklist. | VnVPlan version 2.2 | Commit 182f679<br>Issue #267 |
| Peer | Non-dynamic testing feasibility. | VnVPlan version 2.3 | Commit 194f4d1<br>Issue #270 |
| Peer | Missing list of tables. | VnVPlan updated | Commit fc10209<br>Issue #272 |
| Peer | Missing auxiliary constants definition. | VnVPlan updated | Commit 38c0170<br>Issue #273 |
| Peer | Quantify error measurements in Test-FR-1.2. | WONT FIX | Issue #274 |

Continued on next page

**Table 6 continued from previous page**

| Feedback Source | Feedback Summary | Changes Made | Related Commits/Issues |
|---|---|---|---|
| Peer | Clarify logs in Hardware Diagnostics Tests. | VnVPlan updated | Commit 3049095<br>Issue #275 |
| TA | Missing symbols, abbreviations, and accronym table | VnVPlan version 2.5 | Commit 286836e<br>Issue #337 |
| TA | Explain the "Why" of primary objectives | VnVPlan version 3.4 | Commit fc627ba |
| TA | Rationale of accuracy threshold | VnVPlan version 3.4 | Commit fc627ba |
| TA | Formatting issues: Too much white space | VnVPlan version 3.4 | Commit fc627ba |
| TA | Remove implementation details from test cases. | VnVPlan version 3.4 | Commit fc627ba |
| TA | Removing unnecessary test deails. | VnVPlan version 3.2 | Commit fc627ba |
| Professor | Update Extras. | VnVPlan version 3.0 | Commit a7a739e<br>Issue #404 |
| Team | Update Clang Static Analyzer tool. | VnVPlan version 3.1 | Commit 538a19f<br>Issue #233 |
| Team | Update Code Coverage tool. | VnVPlan version 3.5 | Commit 43ab53c<br>Issue #520 |
| Team | Test-FR-2.3 error handling test procedure updated. | VnVPlan version 4.0 | Commit ccd3793 |
| Team | Update tests with new classification classes. | VnVPlan version 4.1 | Commit 6edf8cc<br>Issue #642 |
| Team | Specifying Software Theory Report. | VnVPlan version 4.2 | Commit f0e4e4b<br>Issue #664 |

Table 6: VnV Plan Feedback Tracking and Implementation

| Feedback Source | Feedback Summary | Changes Made | Related Commits/Issues |
|---|---|---|---|
| Peer | Clarification on code coveration exclusion. | VnVReport version 2.0 | Commit 13790b1<br>Issue #601 |
| Peer | Unclear supporting modules. | VnVReport updated | Commit fa45769<br>Issue #602 |
| Peer | Clarify hardware platform comaparison. | VnVReport updated | Commit be7cec1<br>Issue #604 |
| Peer | Stakeholder inconsistency across tests | VnVReport version 2.10 | Commit 59afc1b<br>Issue #606 |
| Peer | Explain failed unit tests. | VnVReport version 2.8 | Commit c7ee32c<br>Issue #603 |

Continued on next page

**Table 7 continued from previous page**

| Feedback Source | Feedback Summary | Changes Made | Related Commits/Issues |
|---|---|---|---|
| Peer | Vague sound classification accuracy results. | VnVReport version 2.9 | Commit 1f7c57d Issue #599 |
| Team | Removal of microphone different sample size. | VnVReport version 2.2 | Commit 62eb329 |
| Team | Updated classifcation class tests. | VnVReport version 2.3 | Commit e001b72 Issue #624 |
| TA | Missing reflection response on CI/CD. | VnVReport version 2.4 | Commit a0cb47a Issue #654 |
| TA | Unit test linking are broken. | VnVReport version 2.5 | Commit a0cb47a Issue #654 |
| TA | Formatting of unit test module traceability can be improved. | VnVReport version 2.6 | Commit a0cb47a Issue #654 |
| TA | Specifying quality of NFR. | VnVReport version 2.7 | Commit a0cb47a Issue #654 |

Table 7: VnV Report Feedback Tracking and Implementation

# 3 Extras

The team produced two supplementary extras reports. The first focuses on the theoretical foundations of the software algorithms used in the project, including the Direction of Arrival algorithm and audio classification. It explains how these algorithms were developed and validated without physical hardware by leveraging a Python based microphone room simulation library. The report further details the process of experimentation, refinement, and eventual selection of algorithms for deployment on the microcontroller. The report is located in docs/Extras/Hardware/SoftwareReport.pdf

The second supplementary report documents the hardware used in the project. It outlines the theory behind of key hardware components as well as some of their specification. It also explains how knowing the hardware theory helped supported and constrained software design decisions. The report is located in docs/Extras/Hardware/HardwareReport.pdf

# 4 Design Iteration (LO11 (PrototypeIterate))

The final design was reached through an iterative process driven by hardware readiness, user feedback, and alignment with the secondary stakeholder needs. In the initial implementation presented at the Proof of Concept demo, the software algorithms (Direction of Arrival and audio classification) were developed entirely in a simulated environment using a Python microphone room simulation library. This approach was chosen because the hardware platform was not yet stable enough for reliable development. Simulation enabled rapid experimentation, validation of algorithm feasibility, and early performance comparisons of different implementations without being constrained by incomplete hardware. On the hardware side, the system consisted of four microphones connected to a microcontroller, capable of synchronous audio sampling, figure 1. While this setup demonstrated that individual software and hardware components worked as expected, it did not yet meet the stakeholder's need for an integrated, usable system.

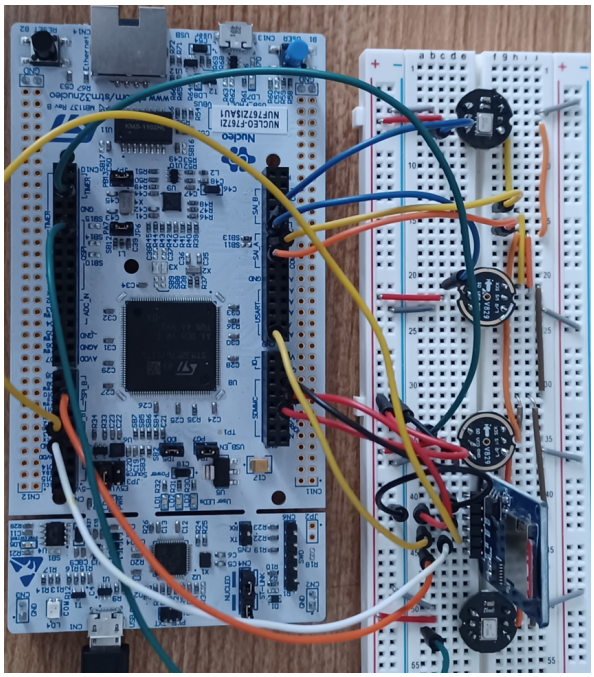

Figure 1: Hardware set-up for Proof of Concept.

As development progressed, the design evolved toward full system integration. The software was migrated from Python to C/C++ to be compiled onto the microcontroller. In parallel, the hardware underwent significant refinements, including improved microphones, more reliable wiring, the addition of bluetooth connectivity, and eventual integration into a custom PCB, figure 2. A key design decision that enabled this parallel evolution was the adoption of a layered software architecture. By decoupling hardware and software components, the team was able to continue developing and testing algorithms independently of hardware readiness using mocks in unit tests. This reduced integration risk and ensured continuous progress toward a deployable system.

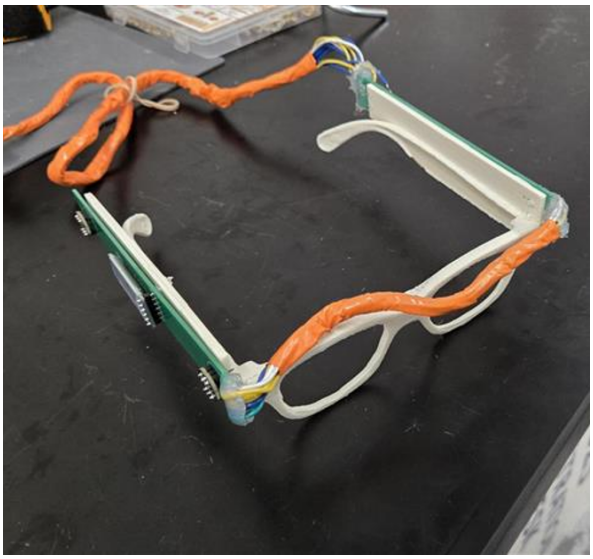

Figure 2: Final hardware.

The user interface design was also refined based on usability considerations. The primary stakeholders are individuals who are hard of hearing. Although direct testing with this group was not feasible, secondary stakeholders (e.x. individuals wearing earplugs or headphones) were asked to participate in user testing. Early UI designs were found to be overly intrusive, presenting excessive information that interfered with users' daily activities, figure 3. This violates the non-functional requirement for a non-intrusive display, NFR6.2. As a result, the interface was simplified to present only essential information in a less disruptive manner, figure 4.

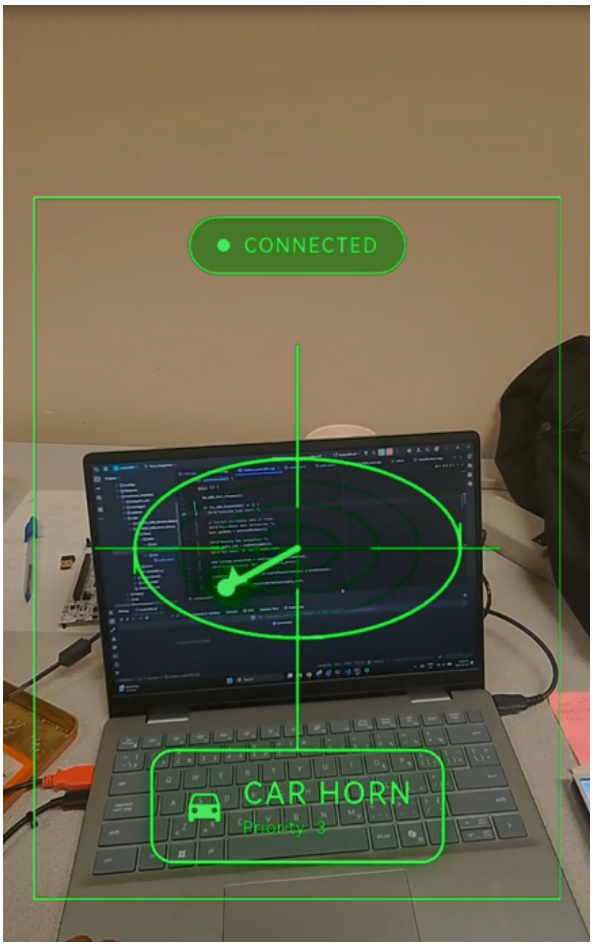


Figure 3: Initial UI before user testing.

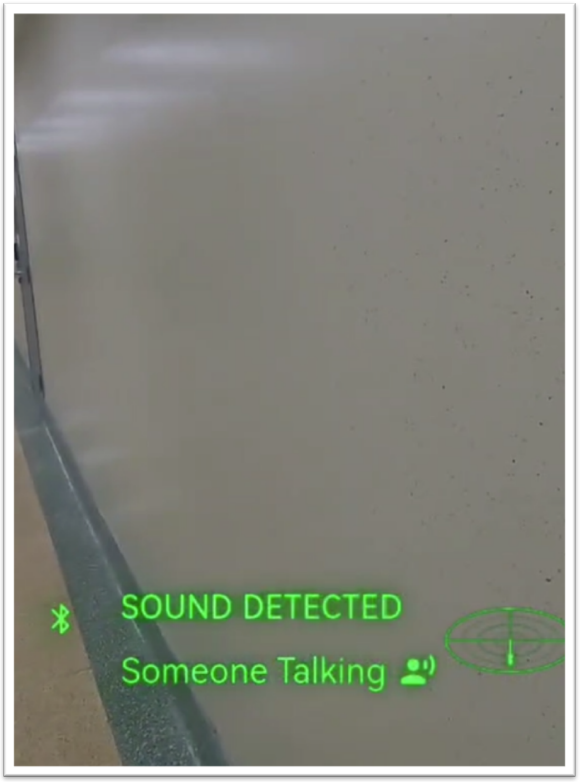


Figure 4: UI after user testing.

Finally, the audio classification component was refined to better reflect meaningful real world use cases. The initial version classified sounds such as car horns, jackhammers, and sirens. However, both user feedback and team evaluation indicated that some of these sounds were less relevant to the core goal of improving situational awareness. The classification categories were therefore updated to include sirens, fire alarms, and human talking, which are more critical for safety and everyday awareness. This required retraining and modifying the classification module, but resulted in a system that better serves the stakeholders' priorities.

# 5 Design Decisions (LO12)

## 5.1 Limitations

The primary limitation of the system was the restricted computational power and memory capacity of the microcontroller. These limitations required careful management of memory allocation and algorithmic complexity to ensure real time performance without any faults. As a result, the team selected lightweight and efficient algorithms. For Direction of Arrival, the GCC-PHAT algorithm was chosen due to its relatively low computational complexity, $O(N*log(N))$, and suitability for embedded systems. More complex alternatives were avoided as they would exceed available resources. Similarly, the audio classification module was limited to three classes to remain within memory constraints. This excluded more advanced approaches, such as neural networks, which would require significantly more memory and processing power than the platform could support.

### 5.2 Constriants

Several constraints from the SRS [1] significantly influenced the design. NFR9.1 specified that the system must operate as a closed system, meaning no external devices or services could be used. This constraint was motivated by security considerations. As a result, this constraint eliminated the possibility of offloading computation to external systems or using external APIs, further reinforcing the need for efficient, on-device algorithms.

Another critical constraint was the requirement for synchronization across all microphones for accurate DoA operation. This influenced both hardware and software design choices. The team selected a microcontroller capable of synchronous audio sampling and designed the wiring and software clock configuration to ensure all microphones operated on a shared timing source. More detailed design decision from microphone synchronization are described in section Microphone Synchronization from the Hardware report (Extras) from this constraint.

### 5.3 Assumptions

The team made several assumptions during development, primarily related to the evolving nature of the hardware. Given limited prior experience with hardware design, it was assumed that components and configurations would change as the project progressed. To mitigate the risks associated with this uncertainty, the team adopted a layered software architecture that decoupled the core Audio360 processing logic from the hardware through a hardware abstraction layer. This design decision allowed software development to proceed independently of hardware changes. This approach reduced development delays and ensured that the system could adapt to hardware modifications without requiring redesign of the core algorithms.

## 6 Economic Considerations (LO23)

[Is there a market for your product? What would be involved in marketing your product? What is your estimate of the cost to produce a version that you could sell? What would you charge for your product? How many units would you have to sell to make money? If your product isn't something that would be sold, like an open source project, how would you go about attracting users? How many potential users currently exist? —TPLT]

## 7 Reflection on Project Management (LO24)

### 7.1 How Does Your Project Management Compare to Your Development Plan

The team generally followed the development plan, with some practical adjustments made to accommodate academic workload and evolving project needs.

The meeting plan originally specified meeting twice a week for team meetings and biweekly meetings with the supervisor. In practice, the team met approximately once per week, which differed slightly from the plan but still ensured consistent progress throughout the project. Meetings with the supervisor were less frequent than initially planned. However, this was a deliberate and mutually agreed adjustment with the supervisor. The team opted to schedule meetings on an as needed basis, primarily when guidance or feedback was required. During Revision 0, the supervisor expressed satisfaction with the team's progress, and efforts towards Rev 1 were focused more heavily on implementation and testing rather than frequent check-ins. Despite these deviations, the team remained ahead of schedule throughout the project.

The team followed the communication plan to the full extent.

Team roles were largely maintained as planned. While there were periods when individual members were less active due to academic commitments, these situations were communicated in advance. This allowed the team to temporarily redistribute responsibilities and maintain overall productivity without significant disruption.

The workflow plan was mostly adhered to. Branching strategies and commit naming conventions were generally followed, and all changes were integrated through pull requests that underwent review before merging into the main branch. Issue tracking was implemented using a Kanban board. However, the board was not consistently updated across all workflow stages (e.x. "in progress", "review", "complete"). This was primarily due to the overhead associated with maintaining detailed task states alongside academic demands. Despite this, the board remained useful for tracking high-level progress and task allocation.

The team used all major tools and frameworks outlined in the development plan. Some adjustments were made as compatibility issues arose during implementation and the development document was updated accordingly. For example, the initially selected code coverage tool was replaced with an alternative that supported the hardware's compiler.

## 7.2 What Went Well?

Issues were created for nearly all tasks, including hardware related work and user testing. This helped the team consistently track what needed to be completed and reduced the likelihood of tasks being overlooked. It also ensured that contributions not reflected in typical metrics, such as number of commits, were still visible. For example, hardware related efforts often do not produce frequent commits, but were still properly captured and tracked through issues.

Defined team roles also worked well. Each member understood their responsibilities, which helped distribute work more evenly and avoided over reliance on a single individual. It also made communication more efficient, as team members knew who to approach for specific needs. For instance, teammates could refer to the meeting note taker for past discussions, or consult the project lead when resolving conflicting ideas.

In terms of technology, the tools adopted by the team were effective in maintaining code quality with minimal overhead. CI workflows that blocked pull requests on failing builds or tests helped prevent faulty code being merged. Additionally, the use of a C++ static code analyzer and code coverage tools provided continuous feedback on code quality, allowing code problems to be identified and addressed early in development.

## 7.3 What Went Wrong?

Since nearly every task was tracked, a large number of issues were created, which made it difficult to clearly identify priorities. As a result, some critical tasks were occasionally overlooked or delayed due to the overhead of managing many issues. This was mainly because there was no explicit way of encoding priority within the issue tracking system.

In terms of technology, there were initially several incompatible decisions made across the toolchain. This was largely due to limited knowledge of how certain technologies would interact with each other early in the project. However, the team was able to mitigate these issues by identifying incompatibilities early, selecting more suitable alternatives, and updating both the implementation and documentation accordingly.

## 7.4 What Would you Do Differently Next Time?

For future projects, the team would place greater emphasis on aligning the project management plan with realistic workload constraints. In this project, several planning decisions did not fully account for other academic commitments, which led to reduced adherence to processes such as frequent meetings and detailed task tracking. A more effective approach would involve designing a lightweight workflow

that minimizes overhead while still maintaining visibility into progress. For example, automating tasks, such as issue deadline reminders or status updates, would reduce manual effort and help ensure accountability without adding unnecessary burden.

Additionally, the team would explicitly incorporate task prioritization into the issue tracking system. In this project, tasks were tracked but not expliclity prioritized, which made it more difficult to identify critical path items and prevent bottlenecks. Introducing clear priority levels would improve task sequencing and ensure that dependencies are addressed in the correct order.

# 8 Ethics and Equity (LO18)

The team applied the principle of equity and universal design by prioritizing the needs of people who are deaf or hard of hearing, who are the primary stakeholders. These are users whose access to their environmental surrounding is most limited. Rather than designing for the typical SMART glasses user, the team focused on ensuring the device compensates for the limitation of situational awareness by providing real-time feedback on environmental sounds. This ensures that users who cannot rely on hearing can still maintain situational awareness and safety in their daily lives.

Although direct testing with deaf or hard of hearing individuals was not possible, the team engaged secondary stakeholders who experience temporary situational hearing (such as individuals wearing earplugs or headphones). These users acted as proxies to gather and evaluate how effectively the system communicates critical information when auditory input is reduced or absent. This method enabled early validation of the user interface and key design aspects like clarity of visual indicators, ease of interpretation and responsiveness of alerts. Not only did this approach allow the team to iteratively obtain user feedback, but it also ensured the system was designed for a larger group of users. However, this method does not fully capture the lived experiences of deaf or hard of hearing individuals. As such, direct testing with these users remains an essential step for future validation and refinement.

The insights we gathered from the secondary stakeholders were used to refine the system's usability and non-intrusiveness. For example, feedback helped ensure that visual outputs were intuitive and did not overwhelm or distract the user. This aligns with NFR6.2 outlined in the SRS (visualization must be non-intrusive and minimize obstruction). Moreover, by minimizing required user interaction and designing the system to operate passively in real time, the system ensured accessibility for users with varying levels of technical literacy or cognitive load. This design consideration is in line with NFR4.4 of the SRS, which states we must minimize required user interaction during runtime.

By focusing on including the primary stakeholder's needs into the system requirements, the team ensured the system was usable for them as well as the secondary stakeholders. While testing with secondary stakeholders is not the perfect substitute for validating the system with the primary stakeholders, the strategy still allowed the team to iteratively improve the design such that it reduces barriers and promotes equitable access.

# 9 Reflection on Capstone

The capstone project overall was an incredible learning experience. It taught us how different areas of engineering come together in practice, and how much more difficult implementation becomes when software, hardware, and user needs must all be satisfied at the same time. One of the most important lessons from this project was that a technically correct solution is not automatically a useful one. Throughout the project, we had to continually evaluate whether our design choices actually improved situational awareness for our stakeholders, rather than simply demonstrating technical novelty. This became especially clear when we refined the interface after user testing and when we adjusted the classification categories to better reflect sounds that were meaningful in real-world safety contexts.

Those decisions showed us that engineering design is not only about building something that works, but about building something that serves the right purpose for the right users.

Another major lesson was the importance of iteration. Early in the project, many of the system components were developed separately: the audio algorithms were validated in simulation, while the hardware existed as a basic multi-microphone setup. Over time, these parts had to be integrated into a cohesive embedded system. This process taught us that first designs are rarely final designs, and that iterative refinement is not a sign of weak planning, but a necessary part of engineering under uncertainty. We learned to treat prototypes as tools for learning rather than as preliminary versions of the final product. The progression from Python-based simulation to C/C++ deployment on the microcontroller, along with hardware revisions and PCB integration, gave us a much better understanding of how to gradually de-risk a system before full deployment.

The project also deepened our understanding of design trade-offs. Many choices that appear straightforward in theory became constrained in practice by limited memory, limited computational power, and the requirement that the system operate as a closed device. These limitations forced us to think more carefully about algorithm selection, modularity, and scope. For example, instead of choosing the most advanced possible methods, we had to choose methods that were efficient enough to run reliably in real time on the available hardware. Interestingly enough, that also pushed us to learn more as well. Rather than relying on simple API calls to exterenal libraries to acommplish our tasks (eg. OpenAI), we had to build in-house simple algorithms that were tailored to our hardware (simple LDA). This taught us that good engineering is often about selecting the most appropriate solution under constraints, not the most sophisticated one in isolation.

A further takeaway from the capstone was the value of architecture and abstraction. Because the hardware platform was still evolving, it would have been easy for software progress to stall. The decision to separate the core processing logic from hardware-specific details through a layered architecture and abstraction layer allowed development to continue even while the hardware changed. This reinforced the idea that good software design is not only about readability or maintainability, but also about enabling progress when parts of a system are unstable or incomplete. In that sense, the project taught us to see architecture as a practical risk-management tool rather than a purely academic concept.

Compared to previous projects, the team driven timeline (where we independently set our development goals and deadlines) really pushed us to learn project management and the importance of planning. We learned that communication and role clarity have a large impact on productivity, especially in multidisciplinary work where progress is not always visible through code commits alone. At the same time, we saw the limits of over-management: although detailed issue tracking improved visibility, it also created overhead and made prioritization harder. This showed us that effective project management is not about maximizing process, but about finding a level of structure that supports the team without slowing it down. That is a lesson that will be valuable in future design projects and professional engineering work.

Overall, the capstone changed our understanding of engineering from a course-based mindset to a systems mindset. In courses, problems are often well-scoped, assumptions are fixed, and success is measured by correctness. In this project, assumptions changed, hardware evolved, user needs shaped design decisions, and success depended on balancing performance, usability, feasibility, and teamwork at once. The project therefore taught us not only technical content, but also how to operate as engineers in an open-ended environment. Our entire team truly had a blast working on this project!

## 9.1 Which Courses Were Relevant

The courses that were most relevant to the capstone project were:

- Signals and Systems (3MX3)
  - Taught us how to analyze signals and systems.

  - Taught us the Fourier Transforms and how to use it to analyze signals.

- Requirements Engineering (3RA3)
  - Taught us how to engineer requirements for a system.
  - Helped a lot with developing our documentation and project plan.

- Software Design 2 (3DA4)
  - Taught us how to design software for a system.

- Software Architecture (3A04)
  - Taught us how to design larger systems as independent modules and how to develop to Android devices.

- Software Testing (3ST3)
  - Taught us how to test our software with unit tests and integration tests using testing frameworks.

- Integrated Engineering Design (2PX3)
  - Taught us how to design and build a prototype of a system with various fidelity levels.

- Software Development (2AA4 & 2C03)
  - Taught us the fundamentals of software development and general good practices for large system developement.

- Human Computer Interfaces (4HC3)
  - Taught us the fundamentals of human computer interfaces and how to design them with users in mind.
  - Also taught us the method in which to elicit user feedback for user interface design decisions.

## 9.2 Knowledge/Skills Outside of Courses

The skills/knowledge that were needed for the capstone project that were outside of the courses taken were:

- Embedded Software Development
  - Setting up the microcontroller and its peripherals required knowledge outside of course material since it was hardware related.

- Processing Algorithms
  - Required for our Direction of Arrival and Audio Classification algorithms. Though we had the skills of how to implement these algorithms from our classes, we had to learn about the algorithms to do so.

- Flutter Development
  - Required to develop the UI. A new programming language not taught in class was required for this android visual interface.

# References

[1] S. Arulmohan, O. Alam, K. Shah, J. Sharma, and N. Chaudri, "System requirements specification," 2025. [Online]. Available: https://github.com/Team6-SixSense/audio360/blob/main/docs/SRS/SRS.pdf